

Algorithm Design Paradigms

Handout on Binary Search

27 January 2026

§1 Introduction

The method of binary search is characterized by the following game:

Hi-Lo game 1.1. You are playing a number guessing game with a friend. Your friend is thinking of a number between 1 and k , and you have to guess it. After every guess, your friend will say whether your guess is too high, too low, or correct. Prove that you can always guess the number in $\lceil \log_2(k) \rceil$ guesses.

This follows from choosing $\frac{k}{2}$ and then picking the middle element of this smaller range. This would allow us to find the number in $\lceil \log_2(k) \rceil$ queries.

This idea also works for slightly less direct questions:

☰ Hamburgers (Codeforces 371C)

Polycarpus have a fixed hamburger recipe using B pieces of bread , S pieces of sausage and C pieces of cheese; per burger.

At the current moment, in his pantry he has:

- n_b units of bread,
- n_s units of sausage,
- n_c units of cheese.

And the market prices per unit is:

- p_b rubles per bread,
- p_s rubles per sausage,
- p_c rubles per cheese.

Polycarpus's wallet has r rubles.

Each hamburger must be made exactly according to the recipe (ingredients cannot be split or titlestituted), and the store has an unlimited supply of each ingredient.

Write function `burgers :: (Int, Int, Int) -> (Int, Int, Int) -> (Int, Int, Int) -> Int -> Int` which takes (B, S, C) , (n_b, n_s, n_c) , (p_b, p_s, p_c) and r and tells us how many burgers can Polycarpus make.

Examples

burgers (3,2,1) (6,4,1) (1,2,3)	4 = 2
burgers (2,0,1) (1,10,1) (1,10,1)	21 = 7
burgers (1,1,1) (1,1,1) (1,1,3)	1000000000000 = 2000000000001

This question may look like a combinatorics or recursion question, but any of those approaches will be very inefficient.

Let's try to algebraically compute how much money is needed to make x burgers. We can define this cost function as cost times the number of ingredient required minus the amount already in pantry. This will something like:

$$f(x) = p_b \max(0, x \cdot B - n_b) + p_s \max(0, x \cdot S - n_s) + p_c \max(0, x \cdot C - n_c)$$

And now we want to look for maximal x such that $f(x) \leq r$. Well, that can be done using Binary search!

```
burgers (b, s, c) (nb, ns, nc) (pb, ps, pc) r = binarySearch 0 upperBound
```

where

```
-- Cost function f(x)
cost x = let needB = max 0 (x * b - nb)
          needS = max 0 (x * s - ns)
          needC = max 0 (x * c - nc)
        in needB * pb +
          needS * ps +
          needC * pc

upperBound = maximum [b,s,c] + r

binarySearch low high
| low > high = high
| otherwise =
    let mid = (low + high) `div` 2
    in if cost mid <= r
        then binarySearch (mid + 1) high
        else binarySearch low (mid - 1)
```

Idea

An problem is killable via Binary Search if

- There exists a way to query (cheaply) such that half of the search space could be eliminated
- We have an easy way to exclude the eliminated elements from the search space

Note: Often the second property is inherited from an order that can be defined on the search space

Remark

Further notice that the time complexity of the algorithm is $\mathcal{O}(\log_2(S)(q + e(n)))$ where S is the size of search space, q is the time per query and $e(n)$ is the time to exclude $\frac{n}{2}$ elements from the search space.

Please mind the use of $\mathcal{O}$ and not Θ .

§2 Square Root

One of the reasons we are interested in this design technique is as we could do this to find roots of polynomials, especially square roots. How?

While using a raw binary search for roots would be impossible as the exact answer is seldom rational and hence, the algorithm would never terminate. So instead of searching for the exact root, we look for an approximation by keeping some tolerance. Here is what it looks like:

```
-- | Square root by binary search
bsSqrt :: Float -> Float -> Float
bsSqrt n tolerance
  | n > 1      = binarySearch 1 n
  | otherwise = binarySearch 0 1
  where
    binarySearch low high
      | abs (guess * guess - n) <= tolerance = guess
      | guess * guess > n                    = binarySearch low guess
      | otherwise                           = binarySearch guess high
    where
      guess = (low + high) / 2
```

We can use our formula for time complexity. Our search space is from 1 to n with a resolution of the tolerance aka ε . Finally, it is taking $\mathcal{O}(1)$ to query and $\mathcal{O}(1)$ to exclude.

Thus, $\mathcal{O}(\log(\frac{n-1}{\varepsilon})) = \mathcal{O}(\log(\frac{n}{\varepsilon}))$.

The internal implementation sets the tolerance to some constant, defining, for example as `sqrt = bsSqrt 0.00001`

§3 A Weird Example

☰ Carnival (CEIO 2014)

Each of Peter's N friends (numbered from 1 to N) bought exactly one carnival costume in order to wear it at this year's carnival parties. There are C different kinds of costumes, numbered from 1 to C . Some of Peter's friends, however, might have bought the same kind of costume. Peter would like to know which of his friends bought the same costume. For this purpose, he organizes some parties, to each of which he invites some of his friends.

Peter knows that on the morning after each party he will not be able to recall which costumes he will have seen the night before, but only how many different kinds of costumes he will have seen at the party. Peter wonders if he can nevertheless choose the guests of each party such that he will know in the end, which of his friends had the same kind of costume. Help Peter!

Peter has $N \leq 60$ friends and we can not have more than 365 parties (as we want to know the costumes by the end of the year).

TODO. *Full Solution*

§4 A Note on One Sided Binary Search

TODO. *Tessellate ke baad!*

§5 Exercises

☰ Caves (IOI 2013, P4)

While lost on the long walk from the college to the UQ Centre, you have stumbled across the entrance to a secret cave system running deep under the university. The entrance is blocked by a security system consisting of N consecutive doors, each door behind the previous; and N switches, with each switch connected to a different door.

The doors are numbered $0, 1, \dots, 4999$ in order, with door 0 being closest to you. The switches are also numbered $0, 1, \dots, 4999$, though you do not know which switch is connected to which door.

The switches are all located at the entrance to the cave. Each switch can either be in an up or down position. Only one of these positions is correct for each switch. If a switch is in the correct position then the door it is connected to will be open, and if the switch is in the incorrect position then the door it is connected to will be closed. The correct position may be different for different switches, and you do not know which positions are the correct ones.

You would like to understand this security system. To do this, you can set the switches to any combination, and then walk into the cave to see which is the first closed door. Doors are not transparent: once you encounter the first closed door, you cannot see any of the doors behind it. You have time to try 70,000 combinations of switches, but no more. Your task is to determine the correct position for each switch, and also which door each switch is connected to.

☰ Rating Recalculation (Code Forces)

It is well known in the Chess Federation that the boundary for the Grandmaster title is carefully maintained just above the rating of International Master Wupendra Wulkarni. However, due to a recent miscalculation in the federation's new rating system, Wulkarni was mistakenly awarded the Grandmaster title.

To correct this issue, the federation has decided to revamp the division system, ensuring that Wupendra is placed into Division 2 (International Master), well below Grandmaster status.

A simple rule like `if rating <= wupendraRating then div = max div 2` would be too obvious and controversial. Instead, the head of the system, Mike, proposes a more titlele and mathematically elegant solution.

First, Mike chooses the integer parameter $k \geq 0$.

Then, he calculates the value of the function $f(r - k, r)$, where r is the user's rating, and

$$f(n, x) := \frac{1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots + \frac{x^n}{n!}}{e^x}$$

Finally, the user's division is defined by the formula

$$\text{div}(r) = \left\lfloor \frac{1}{f(r-k, r)} \right\rfloor - 1$$

.

Write function `ratingCon :: Int -> Int` to find the minimum k , given Wupendra's rating, so that the described algorithm assigns him a division strictly greater than 1 and GM Wulkarni doesn't become a reality.

Examples

```
ratingCon 5 = 2
ratingCon 100 = 5
ratingCon 200 = 7
ratingCon 2500 = 23
ratingCon 3000 = 25
ratingCon 3500 = 27
```

☰ A puzzle (UVA 10025)

A classic puzzle involves replacing each ? with one can set operators + or −, in order to obtain a given k

$$? 1 ? 2 ? \dots ? n = k$$

For example: to obtain $k = 12$, the expression to be used will be:

$$-1 + 2 + 3 + 4 + 5 + 6 - 7 = 12$$

with $n = 7$

Write function `puzzleCount :: Int -> Int` which given a k tells us the smallest n such that the puzzle can be solved.

Examples

```
puzzleCount 12 = 7
puzzleCount -3646397 = 2701
```

☰ New Bakery(Codeforces)

Bob decided to open a bakery. On the opening day, he baked n buns that he can sell. The usual price of a bun is a coins, but to attract customers, Bob organized the following promotion:

- Bob chooses some integer k ($0 \leq k \leq \min(n, b)$).
- Bob sells the first k buns at a modified price. In this case, the price of the i -th ($1 \leq i \leq k$) sold bun is $(b - i + 1)$ coins.
- The remaining $(n - k)$ buns are sold at a coins each.

Note that k can be equal to 0. In this case, Bob will sell all the buns at a coins each.

Write a function `profit :: Int -> Int -> Int -> Int` Help Bob determine the maximum profit he can obtain by selling all n buns with a being the normal price and b the price of first bun to be sold at a modified price.

Example

```

profit      4      4      5 = 17
profit      5      5      9 = 35
profit     10     10      5 = 100
profit 1000000000 1000000000 1000000000 = 1000000000000000000
profit 1000000000 1000000000      1 = 1000000000000000000
profit      1000      1    1000 = 500500

```

Note

In the first test case, it is optimal for Bob to choose $k = 1$. Then he will sell one bun for 5 coins, and three buns at the usual price for 4 coins each. Then the profit will be $5 + 4 + 4 + 4 = 17$ coins.

In the second test case, it is optimal for Bob to choose $k = 5$. Then he will sell all the buns at the modified price and obtain a profit of $9 + 8 + 7 + 6 + 5 = 35$ coins.

In the third test case, it is optimal for Bob to choose $k = 0$. Then he will sell all the buns at the usual price and obtain a profit of $10 \cdot 10 = 100$ coins.